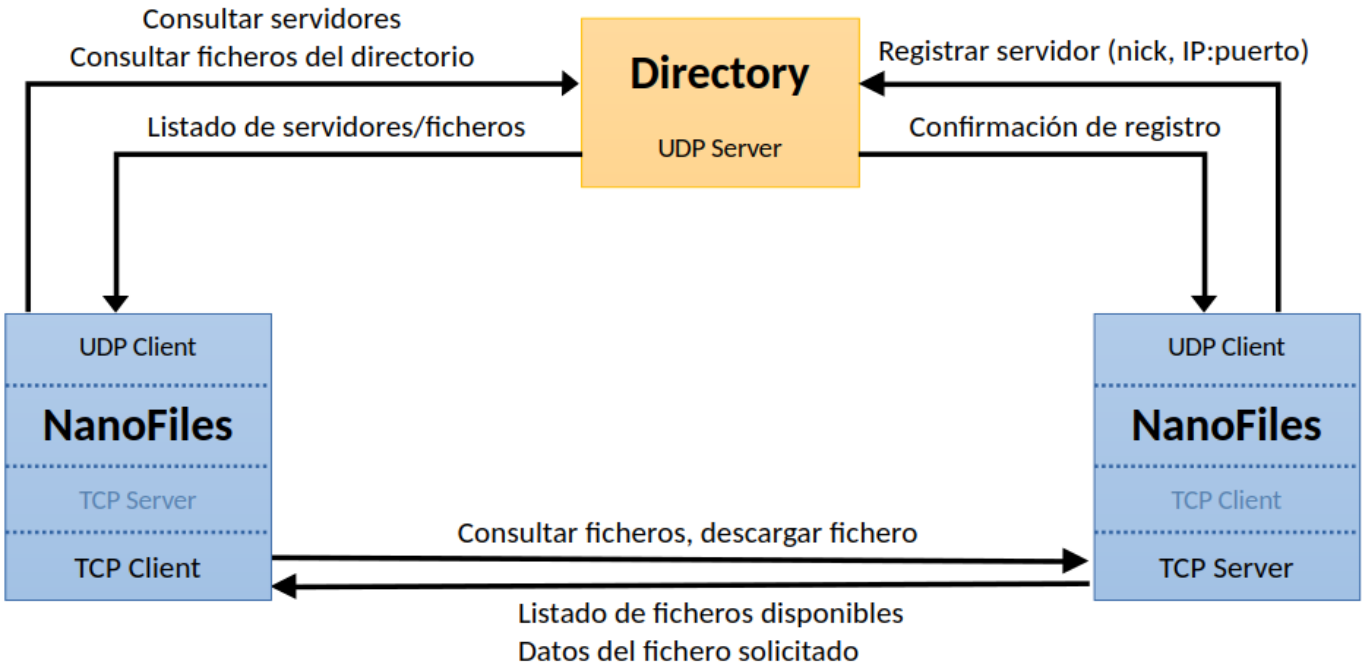
Enunciado del proyecto de prácticas: *nanoFiles*

Redes de Comunicaciones - Curso 2025/26

Visión global y objetivos

En el desarrollo de esta práctica, el alumnado aprenderá a diseñar y programar protocolos de comunicación en Java. La práctica consiste en realizar un sistema de compartición y transferencia de ficheros al que denominaremos *nanoFiles*, y que se describe en este documento.

La siguiente figura muestra un esquema general orientativo del conjunto de componentes de *nanoFiles*:



El sistema *nanoFiles* está formado por un servidor de directorio (programa *Directory*) y un conjunto de *peers* o pares (programa *NanoFiles*). La comunicación en el sistema combina:

- Un modelo cliente-servidor entre cada *peer* y el servidor de directorio.
- Un modelo peer-to-peer (P2P) entre los propios *peers*.

El objetivo global de la práctica es que el alumnado sea capaz de diseñar los protocolos necesarios para intercambiar los mensajes involucrados, así como de implementar el software completo de todos los elementos del sistema.

Especificación de la práctica

Una vez implementada la funcionalidad exigida, un *peer* podrá actuar simultáneamente como cliente y como servidor de ficheros.

En el sistema *nanoFiles*, todos los pares deben ser capaces de comprobar si existe un servidor de directorio que utilice un protocolo de comunicación compatible con el suyo. De esta forma, será posible distinguir el servidor del propio grupo frente a otros servidores que puedan estar ejecutándose simultáneamente en el laboratorio.

La función principal del servidor de directorio es mantener el censo de pares, asociando el *nickname* de cada par con su dirección de escucha. Con esta información, los *peers* pueden conectarse directamente entre sí para consultar y descargar ficheros. Además, el directorio dispone de una carpeta de ficheros propia que puede ser consultada y descargada por cualquier *peer*.

Para que los *peers* puedan intercambiar ficheros, cada uno debe poder lanzar un servidor de ficheros que atienda peticiones de otros pares, proporcionando el listado de ficheros compartidos (nombre, tamaño y hash) y enviando el fichero solicitado, completo o por fragmentos.

Diseño de los protocolos necesarios

Se deberán diseñar dos protocolos de nivel de aplicación:

- Para la comunicación *peer* ↔ directorio, un protocolo confiable semi-dúplex basado en parada y espera que opere sobre UDP.
- Para la comunicación *peer* ↔ *peer*, un protocolo que asuma un transporte confiable como TCP.

Se deberán especificar los siguientes autómatas:

- Servidor de directorio.
- *Peer* como cliente del directorio.
- *Peer* como cliente de otro *peer* servidor de ficheros.
- *Peer* como servidor de ficheros ante otro *peer* cliente.

Respecto a los mensajes:

- Mensajes textuales (*Field:Value*) para la comunicación con el directorio.
- Mensajes binarios para la comunicación entre pares.

Se deja a elección del alumnado la decisión sobre los tipos de mensajes necesarios y la codificación de los campos.

Implementación del software

Se proporciona al alumnado un material de partida que consta de:

- Esqueleto del servidor de directorio.
- Implementación sin terminar de la parte cliente del programa *NanoFiles*, incluyendo:
  - Interfaz de usuario basada en un intérprete de comandos (ya implementado).
  - Lógica del programa cliente (parcialmente implementado).
- Implementación esbozada de la parte servidor del programa *NanoFiles*, incluyendo:
  - Servidor capaz de ejecutarse tanto en primer como en segundo plano.
  - Clase que modela los hilos que pueden ser creados por el servidor para atender a clientes.
- Clases auxiliares:
  - Para la representación de mensajes (sólo esbozadas).
  - Para obtener la base de datos de ficheros compartidos (ya implementada).

El trabajo del alumnado consistirá en completar tanto el programa *NanoFiles* como el programa *Directory* para conseguir que la práctica sea funcional.

Lógica del programa *Directory*

El programa *Directory* quedará a la espera, una vez lanzado, de los mensajes recibidos de los *peers*. El servidor de directorio utilizará el puerto **6868/udp** para recibir mensajes entrantes. Irá actualizando sus estructuras de datos con las peticiones recibidas y, en su caso, responderá con la información solicitada.

El directorio mantiene:

- El censo de pares registrados (*nickname* → `IP:puerto TCP`).
- Los ficheros locales de su carpeta compartida, configurable mediante `-dir <ruta>` (por defecto `dir-shared`).

Con la finalidad de facilitar la depuración, el directorio debería imprimir por consola un breve resumen de los mensajes recibidos y enviados, así como de aquellos mensajes que se han perdido.

El directorio ya incorpora un mecanismo para configurar la probabilidad de pérdida de paquetes mediante `-loss <probability>`, útil para verificar la implementación del protocolo confiable sobre UDP. La probabilidad oscila entre 0 (valor por defecto, no se pierde ningún datagrama) y 1 (se pierden todos).

Lógica del programa *NanoFiles*

El programa *NanoFiles* interactuará con el usuario mediante una línea de comandos. Al ejecutarlo, se puede proporcionar opcionalmente la ruta a la carpeta de ficheros compartidos por ese *peer* (por defecto `nf-`

`shared`, que se creará si no existe). En el escenario habitual de Eclipse, dicha carpeta se creará en el directorio raíz del proyecto Java.

Una vez en ejecución, el programa acepta un conjunto de órdenes. Parte de ellas ya están implementadas y otras deben ser completadas por el alumnado, distinguiendo entre funcionalidad mínima y posibles mejoras.

Órdenes disponibles en el programa *NanoFiles*

La secuencia correcta en la cual se pueden teclear estas órdenes, o las situaciones en las que algunas no están permitidas, dependerá del autómata diseñado. Se deberá controlar dicho orden.

Órdenes locales (sin comunicación)

Estas órdenes ya están implementadas y no implican intercambio de mensajes:

- `help`: muestra ayuda sobre las órdenes soportadas.
- `myfiles`: muestra los ficheros de la carpeta compartida por este *peer* (nombre, tamaño y hash).
- `nick <nickname>`: sustituye el *nickname* aleatoriamente asignado a este *peer* al iniciar el programa, por el que se pasa como argumento al comando. No es posible cambiar el *nickname* una vez que el *peer* ha lanzado el servidor de ficheros y se ha registrado en el directorio.

Órdenes de interacción con el directorio (UDP)

Estas órdenes implican comunicación con el servidor de directorio por UDP, utilizando el protocolo confiable diseñado por el alumnado:

- `ping`: comprueba que el directorio está activo y usa un protocolo compatible con el del *peer*.
- `dirfiles`: muestra los ficheros almacenados en el propio directorio.
- `dirdl <hash_substring>`: descarga desde el directorio un fichero cuyo hash contenga la subcadena indicada. El fichero se guarda con el mismo nombre que tenía en el directorio.
- `peers`: muestra los pares registrados en el directorio con su *nickname* y dirección de escucha (`IP:puerto TCP`).

Órdenes relacionadas con el servidor del *peer*

Estas órdenes afectan al estado del propio *peer* como servidor de ficheros:

- `serve`: lanza un servidor de ficheros que escuchan en un puerto TCP, y acto seguido da de alta en el directorio la IP y el puerto TCP de escucha, asociándolo al *nickname* del par. El servidor del *peer* se ejecuta en segundo plano (en un nuevo hilo creado a tal efecto).
- `quit`: cierra la aplicación. Si el servidor estaba activo, se detiene y se solicita la baja al directorio.

Órdenes de interacción con otros *peers* (TCP)

Estas órdenes implican comunicación directa con otros *peers* por TCP:

- `peerfiles <peer_nickname>`: consulta la lista de ficheros (por TCP) que comparte el *peer* indicado.

- `peerdl <peer_nickname|*> <hash_substring>`: descarga desde uno o varios *peers* (por TCP) un fichero identificado por una subcadena del hash y lo guarda con el mismo nombre que tenga en el/los servidor(es). Se puede indicar un *nickname* concreto o usar `*` para descargar desde todos los pares que tengan el fichero dado.

Con la finalidad de que el alumnado pueda depurar el correcto funcionamiento de sus servidores de ficheros, se sugiere imprimir un resumen de los mensajes recibidos y enviados.

Funcionalidad obligatoria a implementar

Ninguno de los programas entregados está concluido. Entre otras tareas, el alumnado deberá:

- Implementar el formato de mensajes diseñado para que los datagramas UDP intercambiados con el servidor de directorio contengan mensajes adecuadamente formateados.
- Implementar el formato de mensajes diseñado para que los segmentos TCP intercambiados entre un *peer* cliente y un *peer* servidor contengan mensajes adecuadamente formateados.

Con respecto a los requisitos mínimos para que un proyecto de prácticas se considere satisfactorio, será necesario implementar al menos la siguiente funcionalidad:

1. **Comando `ping`**: contactar con el servidor de directorio para comprobar que está a la escucha y que utiliza un protocolo compatible con el del *peer*.

2. **Comando `dirfiles` básico**: mostrar la lista de ficheros almacenados en el directorio. El listado debe indicar para cada fichero su nombre, tamaño y hash. Se puede asumir que el listado puede acomodarse en un único datagrama.

3. **Comando `peers`**: mostrar el censo de *peers* registrados como servidor en el directorio. El listado debe indicar para cada par su *nickname*, IP y puerto TCP de escucha.

4. **Comando `serve`**: lanzar un servidor de ficheros que escuche conexiones en un puerto TCP, y registrar dicha dirección de escucha (IP y puerto) en el directorio, asociándola al *nickname* del *peer*. Se deja a discreción del alumnado cómo resolver el caso de que el *nickname* ya esté previamente registrado. En todo caso, el servidor se debe ejecutar en un hilo distinto al principal para que sea posible continuar utilizando el programa a través del *shell* e interaccionando como cliente tanto con el directorio como con otros *peers*. Además, el servidor de ficheros debe poder atender a múltiples clientes simultáneamente.

5. **Comando `peerfiles`**: listar los ficheros disponibles en un *peer* servidor de ficheros dado por su *nickname*. El listado debe indicar para cada fichero su nombre, tamaño y hash.

6. **Comando `peerdl <peer_nickname>`**: descargar un fichero de entre los ficheros disponibles en un *peer* dado por su *nickname*. El fichero será identificado por una subcadena del hash. El servidor deberá informar en caso de que la subcadena proporcionada no concuerde con ningún fichero compartido, o bien sea ambigua.

La implementación de esta funcionalidad básica permite alcanzar una calificación máxima de 7 puntos.

Funcionalidad opcional a implementar (mejoras)

Para obtener una mayor calificación en la práctica se propone funcionalidad adicional, cuya puntuación se detalla en la tabla posterior. Junto a las mejoras propuestas, se valorarán también de forma positiva otras mejoras que el alumnado quiera plantear al profesorado.

1. **Comando `dirfiles` ampliado.** Mostrar la lista de ficheros almacenados en el directorio considerando que el listado puede ser arbitrariamente largo y, por tanto, abarcar múltiples datagramas.

2. **Comando `dirdl` básico.** Descargar desde el directorio ficheros de cualquier tipo (de texto o binarios) cuyo tamaño permita acomodar sus datos en un solo datagrama. El fichero será identificado por una subcadena del hash. El servidor deberá informar en caso de que la subcadena proporcionada no concuerde con ningún fichero compartido, o bien sea ambigua.

3. **Comando `dirdl` ampliado.** Descargar desde el directorio ficheros de cualquier tamaño. La descarga debe realizarse correctamente en todo caso, considerando que el nivel de transporte no es confiable (pérdidas, duplicados, reordenación). Esta mejora tiene como requisito la implementación previa de la descarga básica desde el directorio descrita anteriormente.

4. **Comando `peerdl *`.** Descargar un fichero desde todos los pares que lo tengan disponible. El fichero será identificado por una subcadena del hash. Se deberá obtener el censo de pares del directorio y consultar uno a uno para localizar los servidores que tienen el fichero. Tras esto, se alternará la descarga de fragmentos del fichero entre todos los pares que lo comparten.

5. **Comando `quit`.** Mantener permanentemente actualizado el censo de *peers* en el directorio, de forma que si un *peer* que está sirviendo ficheros ejecuta la orden `quit`, deberá comunicarlo al directorio para darse de baja como servidor. Se deberá dejar de servir y terminar la ejecución del programa inmediatamente, independientemente de que se estén sirviendo ficheros a otros pares en ese instante.

La siguiente tabla resume las mejoras propuestas e indica la puntuación adicional que podría obtenerse con la implementación de cada una de ellas.

Mejora	Puntuación máxima
`dirfiles` ampliado	1 punto
`dirdl` básico	1 punto
`dirdl` ampliado	1 punto
`peerdl *`	2 puntos
`quit`	1 punto

Detalles de la entrega

El trabajo que los estudiantes deberán desarrollar es el siguiente:

- Programa cliente/servidor *NanoFiles* y programa servidor de directorio *Directory*.
- Documentación de la práctica.

Instrucciones detalladas:

- Las prácticas deben ser realizadas obligatoriamente por **grupos de dos personas**.
- Los grupos deberán subir al Aula Virtual, a la tarea denominada "Práctica de *nanoFiles*" un archivo comprimido .ZIP que contenga lo siguiente:
    - El **código fuente** en Java del proyecto, listo para ser importado y ejecutado en Eclipse.
    - Los programas `Directory` y `NanoFiles` en formato **JAR ejecutable**. Los ficheros se deben llamar **obligatoriamente** `NanoFiles.jar` y `Directory.jar`, y deben funcionar con **Java 21**.
    - La documentación del proyecto. Debe estar en formato de documento PDF e incluir al menos los siguientes apartados:
        - Introducción.
        - Protocolos diseñados.
            - Directorio:
                - Formato de los mensajes y ejemplos de los mismos.
                - Autómatas cliente y servidor
            - Peer-to-peer:
                - Formato de los mensajes y ejemplos de los mismos.
                - Autómatas cliente y servidor
        - Mejoras implementadas y breve descripción sobre su programación.
        - Capturas de pantalla que muestren mediante Wireshark un intercambio de mensajes con el directorio.
        - Opcional: Enlace a grabación de pantalla mostrando los programas en funcionamiento.
        - Conclusiones.
- **El proyecto entregado debe funcionar en los laboratorios de la Facultad**, en el sistema operativo **Ubuntu**, y en un escenario en el que tanto los *peers* como el directorio se ejecuten en **hosts distintos**.

La fecha tope de entrega será el **día 3 de mayo de 2026 a las 23:55**, a través de la tarea del Aula Virtual creada a tal efecto.

Las entrevistas se desarrollarán preferentemente durante la siguiente semana, en el horario y lugar habitual de las clases prácticas, aunque, de ser necesario se añadirán turnos adicionales. Los estudiantes se incribirán para las entrevistas a través de la herramienta *Apúntate*, que cada profesor publicará con suficiente antelación.